Solix EMPOWER Hackathon 2026

AI / Agent Safety & Security

Radius

One change happens outside your company. Radius shows you everything it touches, and stops the work that is not safe.



Product & Research Brief

User journey · Flow · Wireframes

v1.0 · October 2026

This edition · v1

One gap closed.
One build, split in two.

Version 1.0 said what Radius does with a change. It did not say how a change gets in. This edition decides the input, and it splits the build between two builders who never touch the same file.

WHAT IS NEW, AND WHERE

PAGE	TITLE	WHAT IT ADDS
07a	How a change gets in	One box takes words, a pasted advisory or JSON. Three lanes end in one read-back card.
07b	A change is a selector	The `ChangeEvent` format. Why the unseen change needs no new code.
12a	Wireframe 01b · Enter a change	The box and the card, in three states: exact, ambiguous, unmatched.
24a	Build spec 04b · The intake	Extractor schema, resolver rules, when a change fires on its own, failure modes.
31a	Two builders. One contract.	Where the work splits, and the five rules that keep the builders apart.
31b	Builder A · Engine and intake	Seven tasks. Each one ends in a test.
31c	Builder B · Screens and input	Seven tasks. Each one runs on mock data.
31d	The contract	Nine endpoints, the JSON shapes, and the shared S1 fixture.

The decision in one line

A person types the change in plain words, or pastes it. A model proposes a start node. Code checks it against the graph. The person sees what Radius understood, and fires it with one click.

The model proposes. The code decides. The line on page 23 holds.

WHAT DID NOT CHANGE

Pages 01 to 34 are as they were. A new page carries a letter after the number of the page it follows, such as 07a. Every reference in the old pages still points to the right place.

The graph, the search, the risk rule, the actions and the screens stay the same. One small change goes into the search: it can start from more than one node (page 07b).

The problem

A notice arrives on Monday. Nobody knows what it touches.

A vendor reports a fault in one tool version. Your company uses that tool. The question is simple, and no one can answer it: which work must stop now?

30%

Reported rise in change failure rates. Teams ship changes faster than they map the effects.

60%

Share of companies that cannot stop an agent that behaves badly. The control does not exist.

30×

Less context needed when a graph answers "what depends on this?" instead of a file search.

WHAT PEOPLE DO TODAY

- A person opens a spreadsheet of agents. The spreadsheet is old.
- A person asks each team by chat message. The answers arrive in days.
- A person reads the code. The code shows one repository, not the chain.

The cost of a slow answer

Work continues while the answer is incomplete. A customer reply goes out. A finance feed updates. Each one uses the faulty tool. The company finds this later, in an audit.

What Radius does

Three steps. Seconds, not days.

01 Take the change

A change arrives from outside. Radius accepts three kinds. A fault in a tool version. A data column that becomes personal data. A new rule, such as “use EU models only”.

02 Follow the links

Radius starts at the change. It follows the links in the graph. A tool leads to its agents. An agent leads to its workflows. A workflow leads to the obligations it serves and the outputs it sends.

03 Act on the result

Radius pauses each high risk workflow. It opens an incident. The incident names the cause. It sends each owner one next step in plain words. Low risk workflows continue, with a warning.

WHAT RADIUS IS NOT

Radius is not a chat tool. It is not a dashboard that a person reads once a week. Radius is a control. It stops work, and it tells a person why.

The test at the demo. The organisers send a change that nobody has seen. Radius handles it live, with no new code. The answer comes from the graph, not from a list of known cases.

The graph

Five kinds of node. Four kinds of link.

The graph is small and strict. Each link has one meaning. Radius follows the links in one direction only, from the change outward.



WHY THE GRAPH STAYS SMALL

A large graph is slow to build and hard to trust. Five node kinds. Each answers one question. A judge reads the whole schema in ten seconds.

WHY DIRECTION MATTERS

Radius walks the links backward from the change. The same reverse search finds the packages that a faulty library affects.

WHERE THE GRAPH COMES FROM

Tools and agents	The AI bill of materials. CycloneDX ML-BOM and SPDX 3.0 already carry the tool version and the model.
Workflows and outputs	The workflow configuration. It already names the agents it runs and where it sends the result.
Obligations and owners	The governance catalog. Solix holds the retention rules, the data labels and each owner.

Three things we left out, on purpose

Cost. Model weights. The company org chart. Each one is real and useful. None of them changes the answer to “what must stop now?” A graph that holds everything answers nothing quickly.

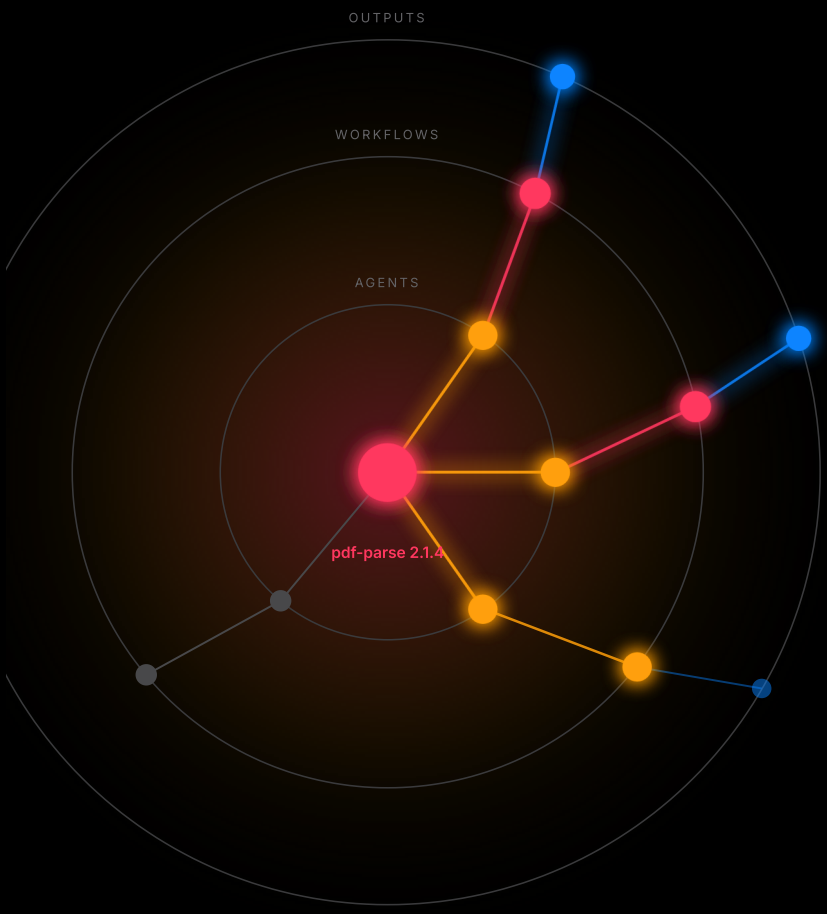
THE GRAPH IS NOT NEW WORK

A company that runs agents already holds this data. It sits in three systems that do not speak to each other. Radius does not ask for a new inventory. It joins the three that exist, and keeps the join current.

The hero moment

One notice in.
The whole blast radius out.

A vendor reports a fault in **pdf-parse 2.1.4**. Radius starts at that tool and follows every link outward.



AFFECTED WORK

PAUSED AUTOMATICALLY

Customer reply
1,204 replies held · Support Ops

Finance feed
GL posting held · Finance Ops

RUNS, WITH A WARNING

Contract review
stays inside the company · Legal

AT RISK

SOX 404 · 1 obligation

3.8s
from notice to paused workflow

3	3	2	1	60
agents	workflows	outputs held	obligation	untouched

Grey nodes are not affected. Radius shows them, so a person can see the size of what is safe.

Why you can trust the list

Every item carries its path.



4

hops from the change to the output. A person can read each one.

0

guesses. No model judges which work is affected.

1 tap

to open any item and see its path in full.

Coverage

Three kinds of change. One engine.

The brief asks for two kinds. We build all three. Each kind starts at a different node. After that, the search is the same.

Kind 1

A faulty tool

A vendor reports a fault in one tool version. The search starts at the **tool** node.

`CVE-2026-4471 · pdf-parse 2.1.4 → 3 agents → 3 workflows`

Kind 2

New personal data

A data steward marks a column as personal data. The search starts at the **data** node, then finds the agents that read it.

`customers.phone → PII → 5 agents → 7 workflows · GDPR Art. 6`

Kind 3

A new rule

A risk officer adds a rule. The search starts at the **obligation** node and runs backward to the agents that break the rule.

`“EU models only” → 4 agents on US models → 6 workflows`

WHY ONE ENGINE MATTERS

A team that writes one rule for each kind of change cannot handle a new kind. Radius has one search. A new kind of change needs one new start node, and nothing else. This is why the live test at the demo is safe for us.

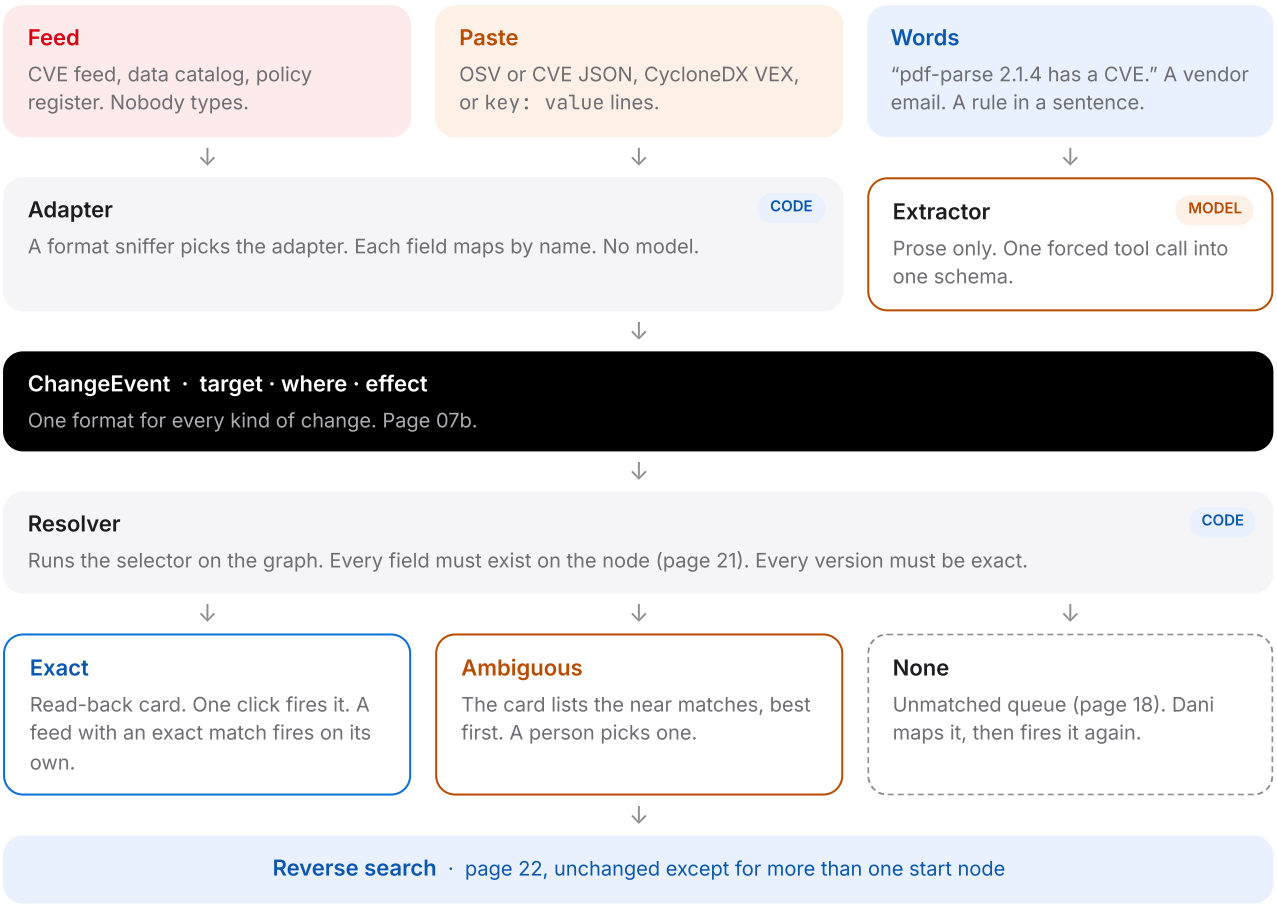
The unseen change

At the demo the organisers send a change that we have not seen. We do not add code. We point the search at the correct start node. The engine does the rest, and the team watches it work.

Input

How a change gets in. One box. One card back.

A change can come from a feed, as a pasted advisory, or as a sentence. All three become the same object. A person sees that object before anything pauses.



Why not a form

A form needs one layout for each kind of change. The unseen change at the demo has no form. The live test fails on the first field.

Why not a chat

We removed the chat box (page 32). A chat answers in turns, and the answer can change between runs. The box takes one message and gives back one card. There is no second turn.

Why not fire on parse

If the parse fires the search, a model picks the start node. Then a model decides which work is affected. That breaks page 23. The card costs one click and keeps the rule.

Input · the format

A change is a selector, not a kind.

Each change names a node type, a condition on that node, and an effect. The three kinds are three examples of one format. The unseen change is one more example, not new code.

```
// one ChangeEvent: Kind 1, typed into the box
{ "id": "chg_0142",
  "kind": "faulty_tool",        // a label only
  "title": "CVE-2026-4471 · pdf-parse 2.1.4",
  "target": "tool",
  "where": { "all": [
    { "field": "name", "op": "eq", "value": "pdf-parse" },
    { "field": "version", "op": "eq", "value": "2.1.4" }
  ] },
  "effect": { "type": "fault", "ref": "CVE-2026-4471",
              "fix": "2.1.6" },
  "mode": "live",                // or "preview"
  "source": { "lane": "words", "raw": "pdf-parse 2.1.4 has…" },
  "parse": { "by": "model", "confidence": 0.96,
             "receipt": "rcpt_88" } }
```

THE GRAMMAR, IN FULL

```
target := tool | data | agent
        | workflow | obligation
where  := { all: [cond…] }
        | { any: [cond…] }
cond   := { field, op, value }
field  := a field of target, page 21
op     := eq | neq | in | not_in
        | contains | prefix
effect := fault | add_label | rule
```

The engine ignores `kind`. It reads only `target` and `where`. The kind is a label for the feed row and the incident.

A rule selects the nodes that break it. "EU models only" becomes `model_region not_in [EU]`. In `preview` mode the same selector is the stretch goal on page 17.

ONE FORMAT, FIVE CHANGES

CHANGE	TARGET	WHERE	EFFECT
Kind 1 · faulty tool	`tool`	`name eq pdf-parse ∧ version eq 2.1.4`	`fault`
Kind 2 · personal data	`data`	`table eq customers ∧ column eq phone`	`add_label pii`
Kind 3 · new rule	`agent`	`model_region not_in [EU]`	`rule`
Unseen · "gpt-x retires Friday"	`agent`	`model_id eq gpt-x`	`fault`
Unseen · "re-check all npm tools"	`tool`	`ecosystem eq npm`	`fault`

One engine change: many start nodes

A selector can match many nodes, such as every npm tool. The search on page 22 puts every match in its first queue at hop 0. Nothing else in the search changes.

```
queue = startIds.map(id => ({ id, hops: 0, path: [id] }))
```

A field we do not hold is a gap, not a guess

Suppose a change names a field that is not on page 21, such as `vendor`. The resolver stops and sends the change to the Unmatched queue, with the missing field named. Nobody guesses which tools a vendor makes.

Research

The method is proven. The subject is new.

Reverse reachability is how a package manager finds the software that a faulty library affects. We apply the same search to agents, workflows and obligations.

WORK	WHAT WE USE	WHERE IT LANDS
Formal Analysis and Supply Chain Security for Agentic AI Skills `arXiv 2603.00195 · Mar 2026`	A skill version is affected if a path exists to a faulty version. The authors use a breadth-first search over the reverse dependency graph — the method `npm audit` uses.	The core search
AgentFlow: Agent Dependency Graphs `arXiv 2607.01640 · 2026`	Builds an agent dependency graph for static analysis. An agent graph can come from code, not only from a person.	Graph build
Graph-Propagated Impact Analysis `arXiv 2606.18855 · 2026`	Propagates impact along a chain of artefacts. Text search alone misses linked items; structure alone misses related items. We use structure, and state the limit.	Known limit
Trust-Aware Multi-Agent Traceability `arXiv 2606.17203 · 2026`	Impact is the set of nodes within h hops of a change. Each link carries a confidence value.	Hop limit
ATAG: Threat Assessment with Attack Graphs `arXiv 2506.02859 · 2025`	Models a multi-agent system as a graph and chains effects between agents.	Risk level
SBOM for AI: Minimum Elements `7 agencies + EC · 12 May 2026`	Names the fields an AI inventory must hold. Radius reads this format as graph input.	Input format

What the research does not do, and one number to remember

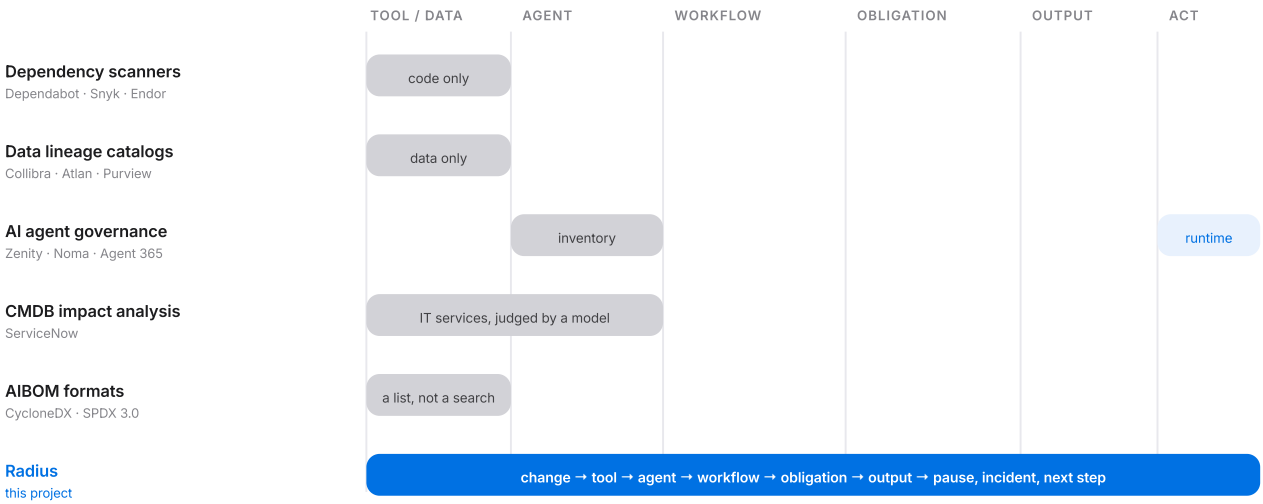
Each paper stops at analysis. None pauses a workflow, opens an incident, or tells an owner what to do. Radius adds the action.

A cross-repository index turns “what depends on this?” from a search of about 6,000 tokens into one graph lookup of about 200. A graph is cheaper, not only clearer. Meta, April 2026. Verify before you quote it.

The market

Everybody stops early.

Five categories of product touch this problem. Each one covers part of the chain. None of them covers the chain from end to end, and then acts.



Lineage looks backward

Catalogs rebuild lineage from query logs. The answer arrives after the event. Radius answers before the next run.

Governance knows agents only

Zenity and Noma hold a live agent inventory and stop a bad action. Neither starts from an outside change and reaches an obligation.

CMDB asks a model

ServiceNow walks the graph, then asks a model to judge the effect. Its own documents say a vague description gives a weak answer.

THE HONEST POSITION

Two products move toward this chain. ServiceNow AI Control Tower and Microsoft Agent 365 each add an agent registry to a large platform. Agent 365 became available on 1 May 2026.

We do not claim that nobody can build this. We claim that nobody ships it today as one search with one action, and that a small team can show it in four seconds.

Where this fits Solix

The Solix Common Data Platform holds the catalog, the lineage and the retention rules. Radius reads that platform as the graph. An obligation is not an idea in Radius. It is a row that Solix already stores.

The people

Three people. One of them should never wait.

Primary user

Dani Alvarez

AI Platform Operations Lead

Dani receives the notice. Dani must answer one question fast: which work must stop now?

Receives a task

Sam Okonkwo

Workflow owner, Finance Ops

Sam owns the finance feed. Sam wants one clear next step, not a report.

Needs the record

Nura Haddad

Risk and Compliance Officer

Nura must show an auditor which obligations the change put at risk, and what the company did.

THE JOURNEY, BEFORE AND AFTER

	Notice arrives	Find what it touches	Decide what stops	Tell the owners	Close
TODAY	Email, 08:40	Ask each team by chat	Guess, or stop everything	Forward the email	No record
Elapsed time					3 days
WITH RADIUS	Feed, 08:40	The list appears	High risk pauses itself	Each owner gets one step	Record kept
Elapsed time					4 minutes

Dani does not read a report. Dani confirms a decision that the system already made.

Sam must never open Radius

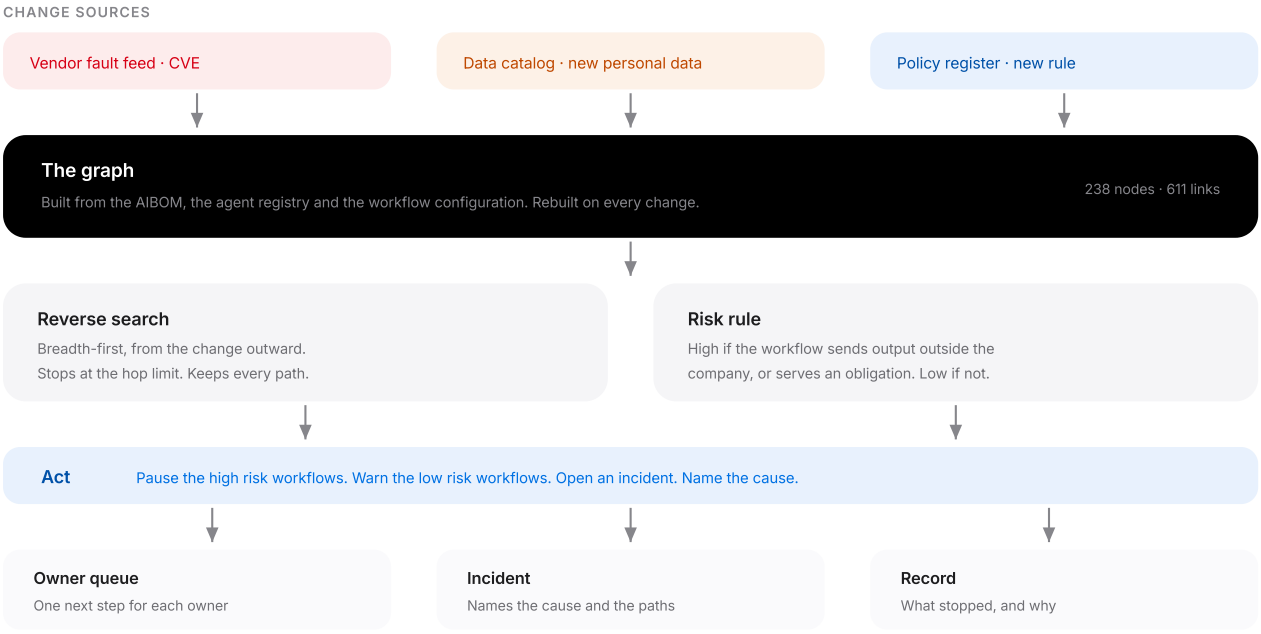
Sam receives one message. The message names the workflow, the cause, and the next step. Sam does not learn a new tool. A control that needs training is a control that people avoid.

WHAT EACH PERSON SEES

Dani	The full list, the paths, and the pause switch.
Sam	One task. One next step. One link back to the cause.
Nura	The obligations at risk, and the record of what happened.

The flow

From a notice to a paused workflow.



No model decides any of this. The search is a graph walk. The risk rule is a condition on the node.

The hop limit keeps it honest

A search with no limit marks the whole company as affected. Radius stops at four hops. A longer path is shown as a separate, lower group. The research calls this the set of nodes within `h` hops.

The risk rule is a condition, not a judgement

A workflow is high risk when it sends output outside the company, or when it serves an obligation. These are properties on the node. A person can read the rule in one line.

WHAT AN INCIDENT SAYS

```
incident   INC-2041
cause      CVE-2026-4471 · pdf-parse 2.1.4
path       tool → Invoice extractor →
Finance feed
risk       high · sends output to finance
action     paused at 08:41
owner      Finance Ops · S. Okonkwo
next       move to pdf-parse 2.1.6
```

Seven lines, always the same seven

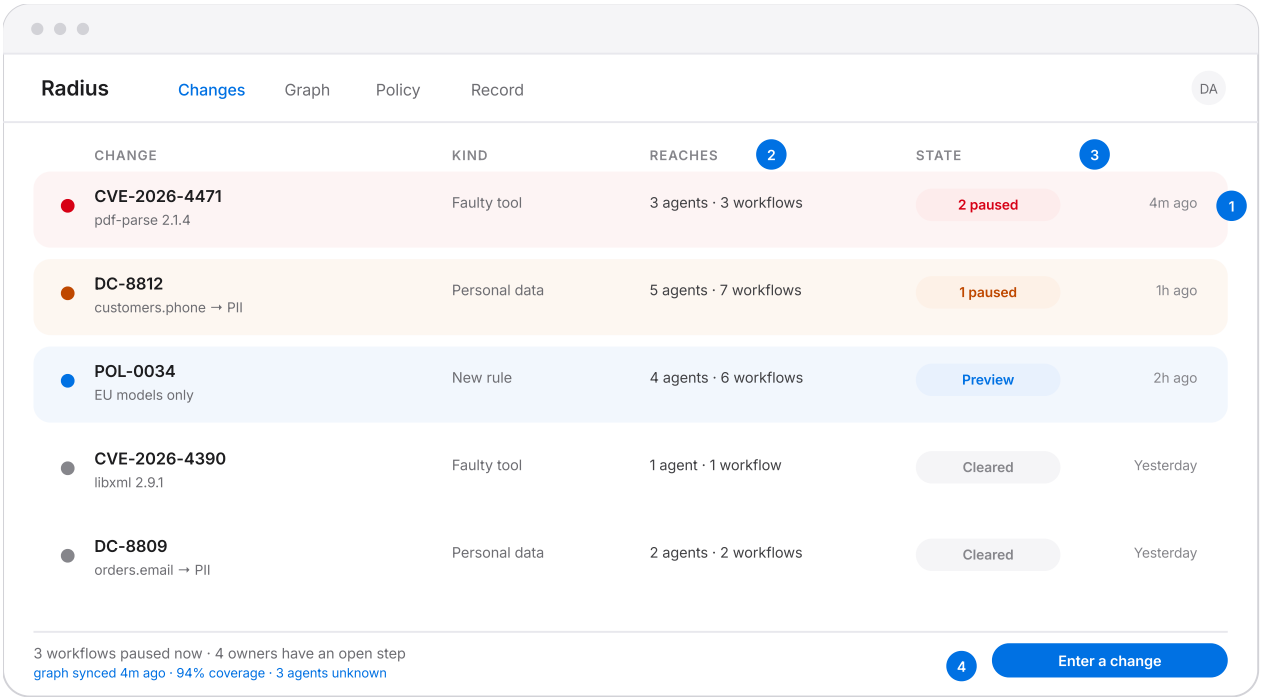
An incident never contains a paragraph. It contains the cause, the path, the risk, the action, the owner and the next step. A person reads it in one look, and a machine can read it too.

The path line is the important one. It is the reason a person believes the other six.

Wireframe 01

The feed. Changes arrive here.

Dani opens one screen. Each row is a change. Each row already shows what the change touches, and what Radius did about it.



1 Colour means state, not noise

Red means a workflow is paused now. Grey means the change is closed. A person reads the screen in two seconds. **Clean & simple**

2 The reach is in the list

A person does not open a row to learn the size of the problem. The count is on the row. **Progressive disclosure**

3 The system acted first

The state column shows what Radius already did. The screen reports a decision. It does not ask for one. **Visibility of system status**

4 One button, for the demo

A person can enter a change by hand. The organisers use this button on stage. **Affordance**

WHY THE FEED IS THE HOME SCREEN

A graph is the engine, not a home screen. A person who opens a graph must decide where to look. A person who opens a list reads it from the top. The graph appears when a person opens one item.

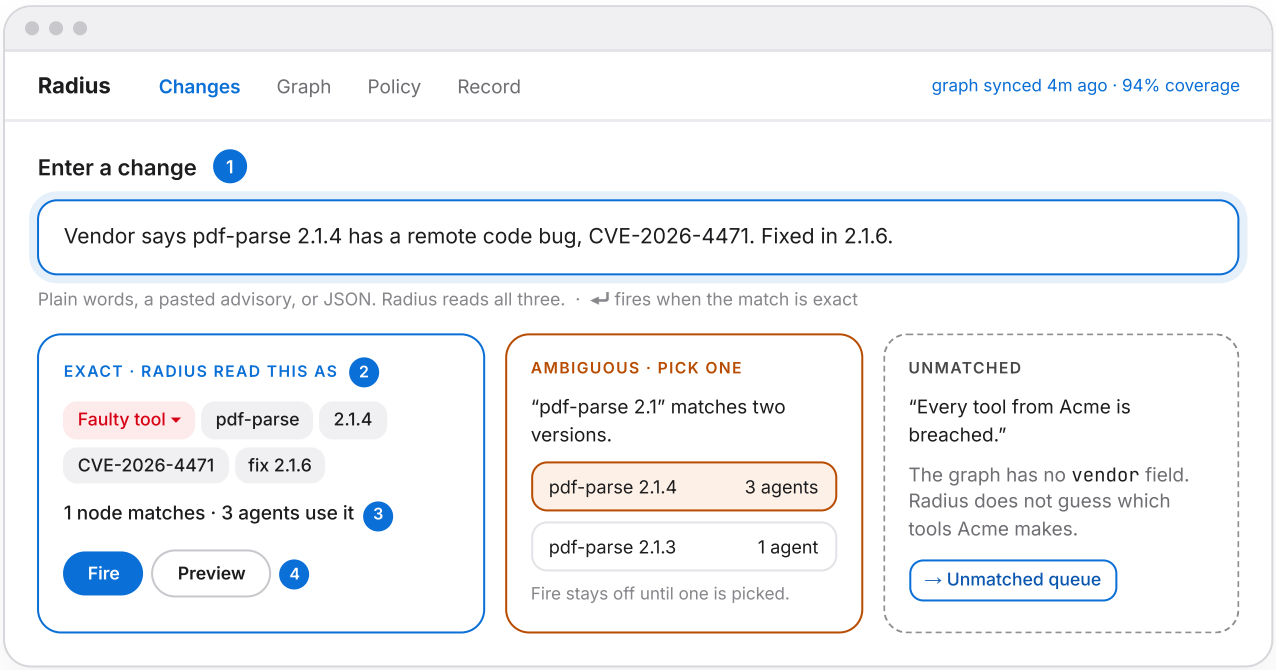
The empty state matters most

On a quiet day this screen shows no red rows. It says how many changes it checked, and that nothing is paused. A control that shows nothing looks broken. This one reports that it worked.

Wireframe 01b

Enter a change. Radius reads it back.

The button on the feed opens one box. A person types or pastes. Radius answers with one card that says what it understood. Nothing runs until the card is right.



1 One box for every kind

The person does not pick a kind first. The kind is an output of the parse, not an input.

Flexibility

2 Chips, not a paragraph

Each part of the change is one chip. To fix a chip, tap it and pick a value that the graph holds.

Recognition over recall

3 The count is on the card

“3 agents use it” shows the match is real before anything fires. A wrong version shows “0 agents”.

Feedback

4 Fire and Preview, side by side

Preview runs the same search with the actions off (page 17). Both buttons have the same shape.

Symmetry

WHY THE CARD IS NOT A CONFIRM DIALOG

A dialog asks “are you sure?”, and people click yes. The card shows what Radius will search for, in the words the graph uses. The person checks a fact, not a feeling.

On stage

The organisers type the unseen change here. The room sees Radius read it back, then fire. That one second shows that a model did not decide the list.

Wireframe 02 · the hero screen

The blast radius. Graph left. Decisions right.



1 The graph is a control, not a picture

A person moves the hop limit and the list changes with it. The person sees the cause and the effect together. **Direct manipulation**

2 The decision is already made

The heading reads PAUSED AUTOMATICALLY. Radius does not ask Dani for permission to make a system safe. **Feedback**

3 Low risk work stays visible

Radius shows the work that continues. A list of only bad news hides the size of the problem. **Clarity**

4 One way back

Resume all is next to the main action, and it is quiet. A person can undo the pause, and the button does not invite it. **Help people avoid errors**

WHEN THE LIST IS LONG

A wide change can touch forty workflows. Radius groups them by owner and sorts them by risk. The screen shows four groups, not forty cards.

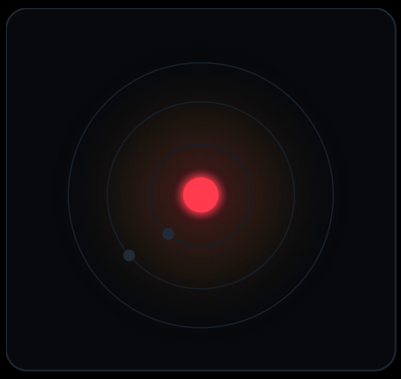
The limit we state out loud

Radius finds what the graph records. A link that nobody entered does not exist for Radius. The research says the same: a structural search misses an item that is related in meaning but has no edge. We show the graph coverage on screen, so a person knows what the answer rests on.

The reveal

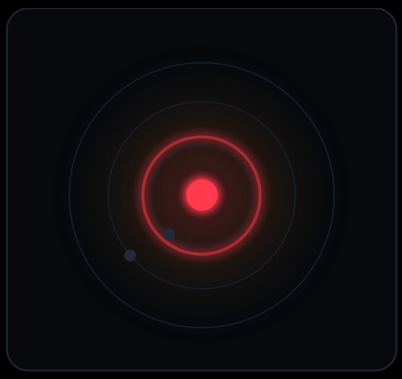
Six frames. Four seconds. Nobody looks away.

The screen starts almost empty, on purpose. One red node and a question. Everything that follows is the answer arriving, one ring at a time, at a speed a person can read.



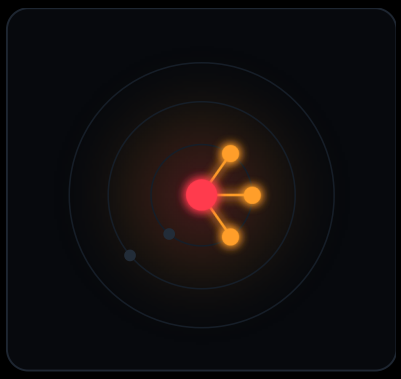
0.00 s The change, alone

Nothing has moved. One node, and a question.



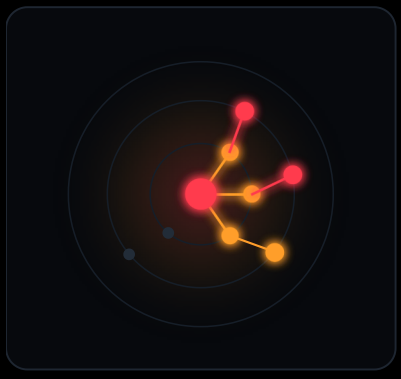
0.12 s The shockwave

A ring leaves the centre. The eye goes where the search goes.



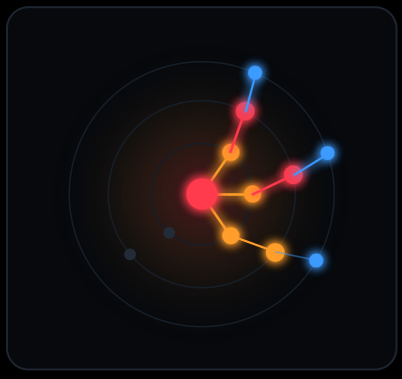
0.52 s Ring one lights

Edges draw first, then the nodes land with a slight overshoot.



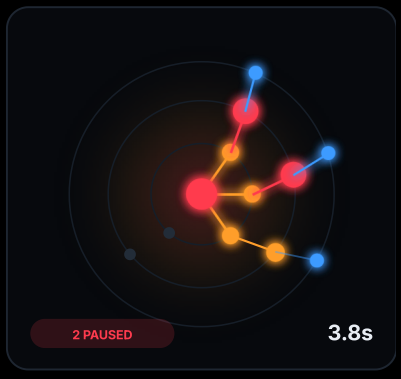
1.50 s Ring two

One beat per hop. The pause between rings is the explanation.



2.40 s Ring three

Outputs and duties. The full reach is on screen.



3.15 s It acts

Two nodes pulse and pause themselves. The counter settles.

Hold the empty frame

Frame one stays for a beat before anything moves. An audience that is given one object will look at it. An audience given six will look at none.

One idea per ring

Agents, then workflows, then what is at stake. Say the ring out loud as it lands. Never explain two rings at once.

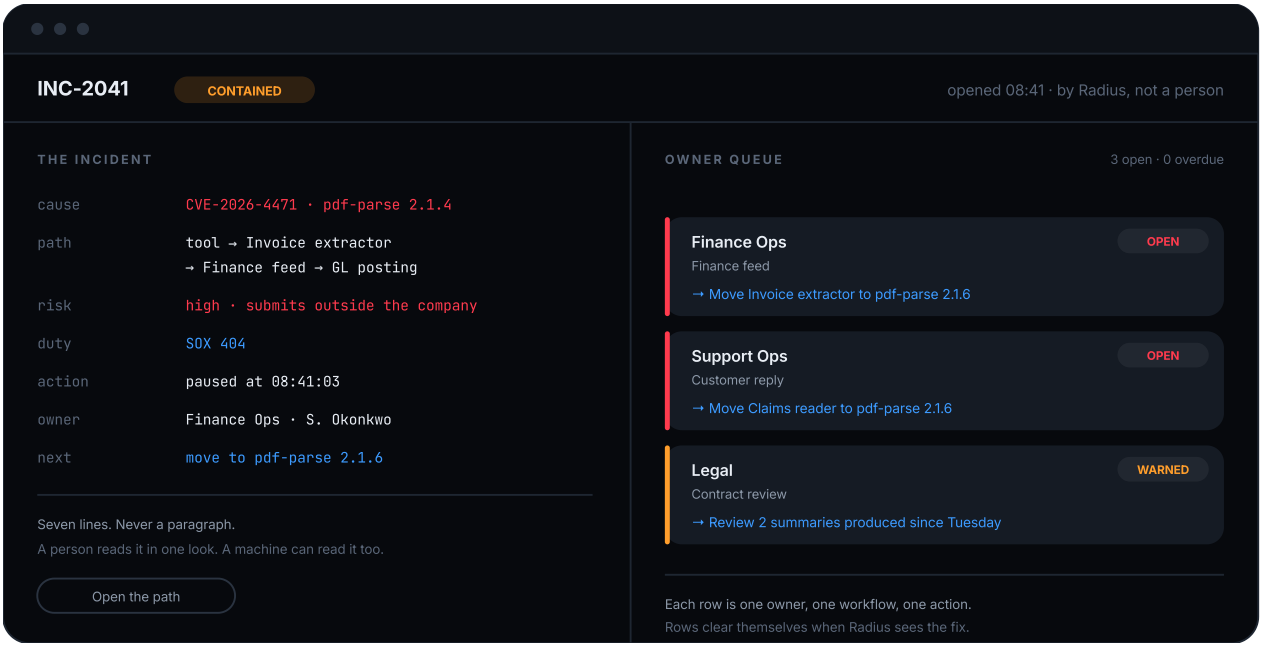
Let it act last

The pause is the final beat, not the first. The room must understand the reach before it sees the system stop the work.

Wireframe 03

The incident, and the owner queue.

The brief asks for two objects by name: an incident that names the vulnerability, and a queue where every owner has one clear next step. Here they are, on one screen.



WHY THE INCIDENT IS SEVEN FIXED LINES

A free-text incident has to be read before it can be used. These seven fields are always present and always in the same order, so a person learns the shape once and a downstream system can parse it without a model.

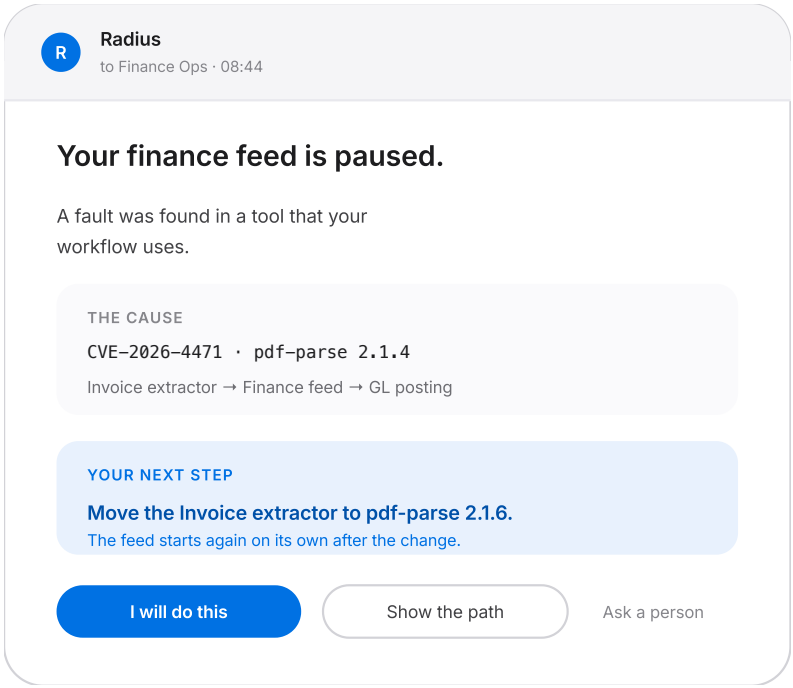
Why a queue, not a dashboard

A dashboard asks a person to find their own work. A queue hands it to them. The row disappears when the fix lands, so the queue empties itself and nobody reports progress by hand.

Wireframe 03

What Sam receives.

Sam owns the finance feed. Sam is not a security engineer. Sam gets one message, in the tool Sam already uses.



One message. One step.

The message does not explain the graph. It names the cause, the path, and the one thing Sam must do. Focus

Plain words, no jargon

“Your finance feed is paused.” Not “workflow WF-212 state transition”. Sam reads it once. Empathy

The way back is automatic

Sam does not report the fix. Radius sees the new version and starts the feed again. Predictability

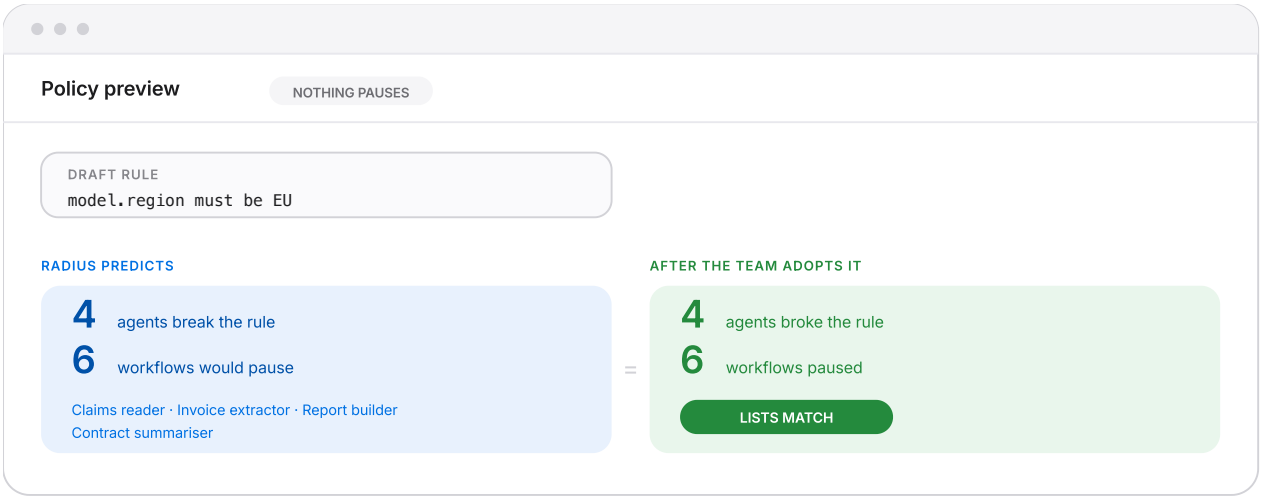
The path is one tap away

Sam can check the reason without a meeting. Trust comes from a proof that is easy to reach. Progressive disclosure

Stretch goal

See what a rule stops, before you adopt it.

A risk officer writes a new rule. The same search runs, but nothing pauses. Radius shows the list that the rule would produce. The team adopts the rule only when the list looks correct.



Why this closes the demo

The team shows the predicted list. The team then applies the rule. The two lists match. This proves that the search is correct, and not only that the screen looks correct.

THE METHOD ALREADY EXISTS

A dry run is standard practice in policy systems. Kubernetes admission control has a `dryrun` mode. AWS IAM Access Analyzer previews the findings of a policy before the team applies it.

Radius applies the same idea to agent work. The new part is the subject, not the mechanism.

When the two lists differ

A difference means the graph is wrong, not the rule. Radius shows the items that only one list holds. That difference is the most useful output the product makes, because it finds a missing link.

A rule is a change

Radius treats a draft rule as the third kind of change. The preview is the same search with the actions switched off. We did not build a second engine for this screen.

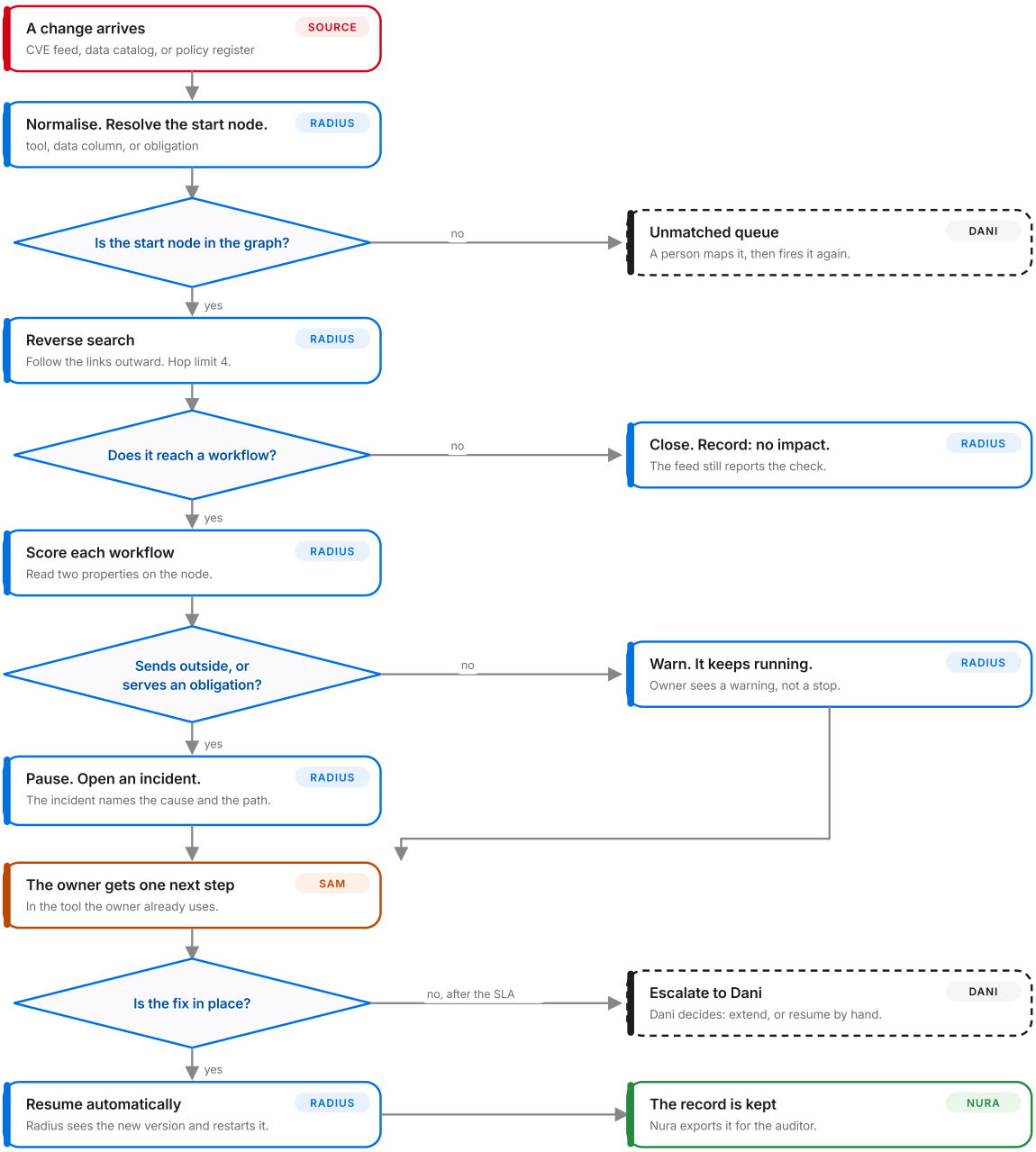
Build this last

Build it only after the two required kinds of change work end to end. A broken preview costs more than a missing one.

End to end

The whole flow, with every branch.

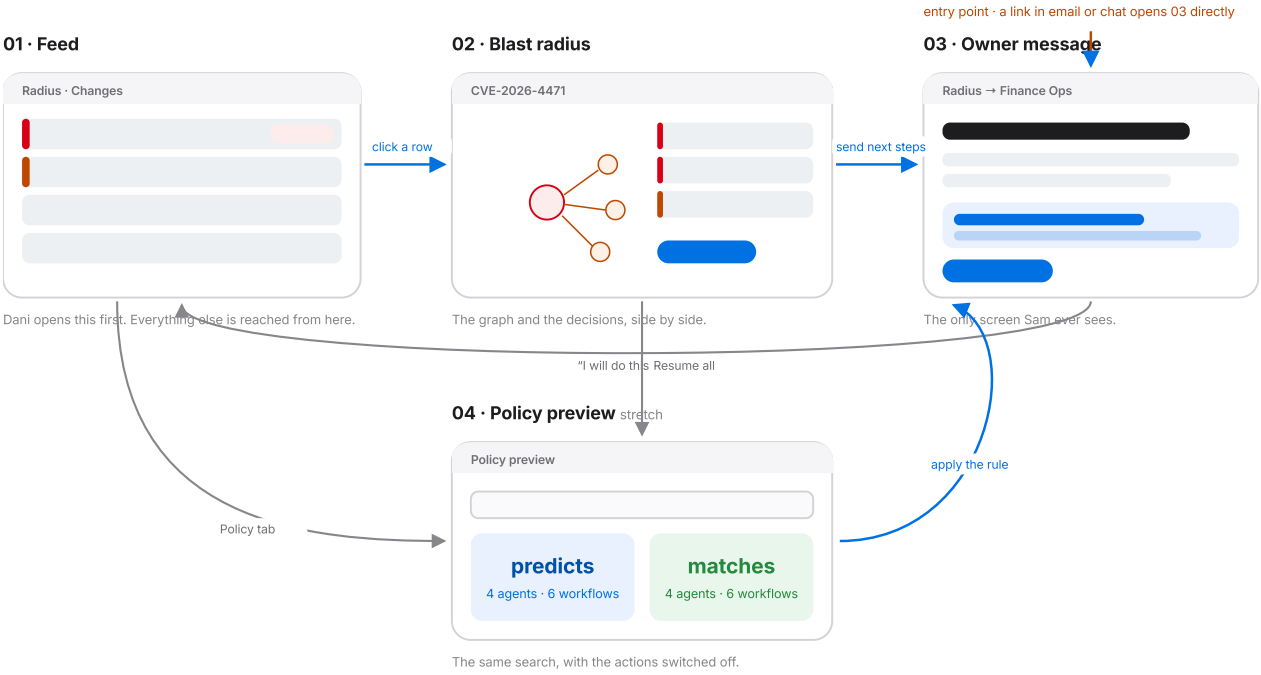
Blue boxes are the system. Dashed boxes need a person. Every exit is shown, including the two where nothing happens.



Wireflow

Four screens. Six moves between them.

Each arrow is a real control on the screen it leaves. Sam never passes through the first two screens, because a link in the message opens the owner view directly.



THE RULE BEHIND THE MAP

Every screen has one job and one main action. A person can reach any screen in two moves from the feed. No screen is a dead end: each one has a visible way back.

Deep link, not a tour

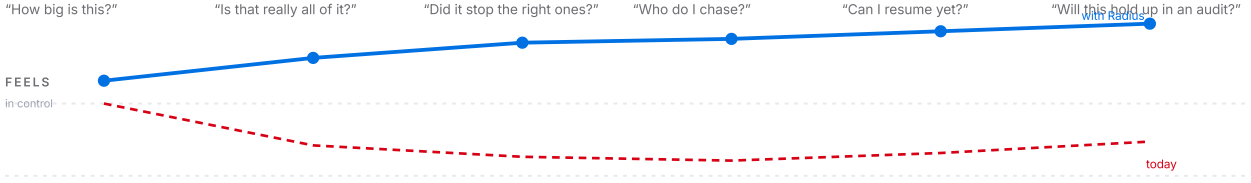
Sam gets a link that opens screen 03 with the task already loaded. Sam never learns the navigation, because Sam never navigates.

User journey map

Dani’s six phases, start to finish.

The red line is the day Dani has now. The blue line is the same day with Radius. The gap is widest at phase 2, where the question “how big is this?” goes unanswered for days.

	1 · Notice	2 · Scope	3 · Contain	4 · Delegate	5 · Track	6 · Close
DOES	The change lands in the feed.	Open the blast radius view.	Confirm what the system paused.	Send the next steps.	Watch the queue empty.	Export the record.
SEES	A red row with a count on it.	The graph and the affected list.	Two workflows already paused.	Three owner tasks, pre-written.	Two done, one open.	Cause, paths and actions, in one file.
THINKS	“How big is this?”	“Is that really all of it?”	“Did it stop the right ones?”	“Who do I chase?”	“Can I resume yet?”	“Will this hold up in an audit?”
PAIN TODAY	No signal at all. An email, maybe.	Ask each team. Wait days.	Guess, or stop everything.	Forward the email and hope.	Chase people by hand.	Write the story from memory.
WHAT RADIUS CHANGES	The row arrives with the reach already counted.	The list is there before Dani asks.	Containment happened before the alert.	Each task names one action and one owner.	Radius sees the fix and resumes on its own.	The record is a by-product, not a task.



THE OTHER TWO JOURNEYS

Sam · workflow owner

`notification → read one step → apply the fix → done`

Four moments, no new tool, about ninety seconds. Sam never opens Radius and never sees a graph. If Sam has to learn something, the design has failed.

Nura · risk and compliance

`obligation view → open the incident → export the record`

Nura enters at the obligation, not the change. The question is never “what broke?” It is “which rules were exposed, and for how long?”

Build spec · 01

The data model.

Every field below is needed by the search or the risk rule. Nothing else is stored.

NODES

TYPE	FIELDS	USED FOR
tool	`id, name, version, ecosystem, owner_id`	Start node for a vulnerability. The version must be exact.
data	`id, table, column, labels[], owner_id`	Start node for a data change. `labels` holds pii, phi, financial.
agent	`id, name, model_id, model_region, owner_id`	`model_region` is what the “EU models only” rule reads.
workflow	`id, name, owner_id, sends_external, state`	`sends_external` is half the risk rule.
obligation	`id, name, authority, owner_id`	SOX 404, GDPR Art. 6, Annex 11. The other half of the risk rule.
output	`id, name, channel, volume`	What is at stake, as a number a person understands.

EDGES

RUNS_ON	agent → tool
READS	agent → data
INVOKES	workflow → agent
SERVES	workflow → obligation
SENDS	workflow → output

Every edge is walked **backward**, except SERVES and SENDS, walked forward from a reached workflow.

```
{
  "id": "wf_finance_feed",
  "type": "workflow",
  "name": "Finance feed",
  "owner_id": "team_finance_ops",
  "sends_external": true,
  "state": "running",
  "edges": [
    { "INVOKES": "ag_invoice_extractor" },
    { "SERVES":  "ob_sox_404" },
    { "SENDS":   "out_gl_posting" }
  ]
}
```

Storage

Hold the graph in memory as adjacency lists, loaded from one JSON file. 238 nodes fit in well under a megabyte and the search finishes in single-digit milliseconds. A graph database earns its place past ten thousand nodes, which the demo never reaches.

Build spec · 02

The search, in full.

A breadth-first walk over the reverse graph, with a hop limit and a kept path. This is the whole engine. There is no model in it.

```
// returns every node reachable from the change, with the path that reached it
function blastRadius(graph, startId, maxHops = 4) {
  const seen = new Map()  // id → { hops, path }
  const queue = [{ id: startId, hops: 0, path: [startId] }]

  while (queue.length) {
    const cur = queue.shift()
    if (seen.has(cur.id)) continue  // first path wins: it is the shortest
    seen.set(cur.id, cur)
    if (cur.hops ⩾ maxHops) continue

    // backward: who points AT this node?
    for (const n of graph.incoming(cur.id, ['RUNS_ON','READS','INVOKES']))
      queue.push({ id: n, hops: cur.hops + 1, path: [...cur.path, n] })

    // forward, only from a workflow: what does it serve and send?
    if (graph.type(cur.id) ⩵ 'workflow')
      for (const n of graph.outgoing(cur.id, ['SERVES','SENDS']))
        queue.push({ id: n, hops: cur.hops + 1, path: [...cur.path, n] })
  }
  return seen
}
```

THE RISK RULE

```
risk(wf) = wf.sends_external
      || wf.serves.length > 0
   ? 'high' : 'low'
```

Two booleans. No threshold to tune, no training data, no drift. A person reads this line once and can predict every verdict the system will give.

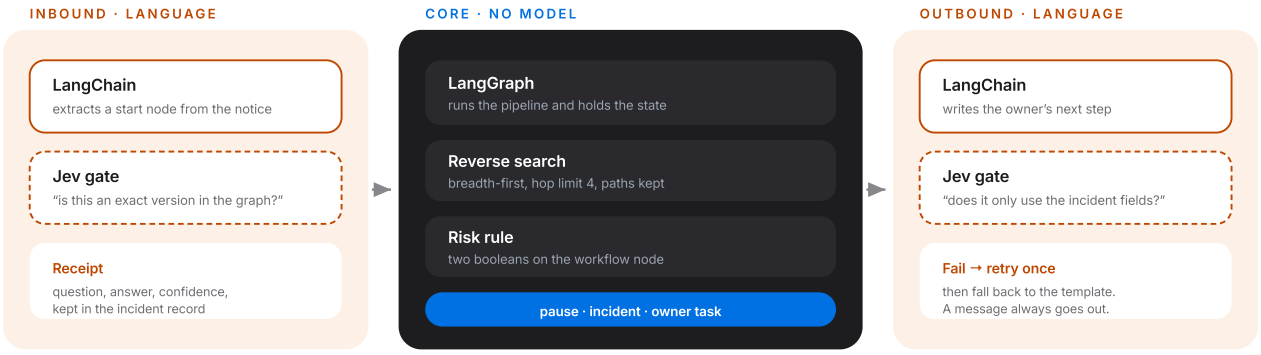
PROPERTIES THAT MATTER

Shortest path	The queue is FIFO, so the first path to reach a node is the shortest. Keep it; drop the rest.
Hop limit	Four. Without it a wide change marks the whole company as affected, which is true and useless.
Cost	`O(V + E)`
Determinism	Sort each adjacency list by id. The same change then returns the same list in the same order, every run.

Build spec · 03

Where the models sit — and where they must not.

Language is messy at the edges of this system and absent in the middle. A model reads the incoming notice. A model writes the outgoing message. Between them, nothing is decided by a model.



The line we never cross

No model decides which work is affected, and no model decides what pauses. If a model fails, the blast radius is unchanged.

Why a model at all

A CVE advisory is prose. A vendor email is prose. Turning either into `pdf-parse@2.1.4` is extraction, which a model does well and a regular expression does badly.

Why Jev between them

Jev answers a narrow closed question and leaves a receipt. The pattern is the one its own harness describes: a model proposes, Jev answers, **code decides**.

What we verify

Jev’s published injection benchmark is third party and self-reported. We use it as a second opinion with a receipt, never as the thing that decides.

Build spec · 04

The LangGraph pipeline.

Every diamond in the end-to-end flow is a conditional edge. Every box is a node. The checkpointer gives the incident record for free, because the state of each run is already durable.

```
class RadiusState(TypedDict):
    raw: str               # the notice as it arrived
    start_node: str | None
    reached: dict          # id → {hops, path}
    verdicts: list[dict]
    receipts: list[dict]   # every Jev answer, kept

g = StateGraph(RadiusState)
g.add_node("extract",    extract_change)     # LangChain
g.add_node("gate_start", jev_gate_start)     # Jev + receipt
g.add_node("triage",     human_triage)      # interrupt()
g.add_node("search",     blast_radius)      # pure BFS
g.add_node("score",      apply_risk_rule)    # pure
g.add_node("act",        pause_and_open)     # side effects
g.add_node("compose",    write_next_steps)   # LangChain
g.add_node("gate_msg",   jev_gate_message)   # Jev + receipt
g.add_node("dispatch",   send_to_owners)

g.add_edge(START, "extract")
g.add_edge("extract", "gate_start")
g.add_conditional_edges("gate_start", resolved,
    {"yes": "search", "no": "triage"})
g.add_edge("triage", "search")
g.add_conditional_edges("search", any_workflow,
    {"yes": "score", "no": END})
g.add_edge("score", "act")
g.add_edge("act", "compose")
g.add_conditional_edges("gate_msg", msg_ok,
    {"yes": "dispatch", "no": "compose"})  # one retry

app = g.compile(checkpointer=SqliteSaver("radius.db"))
```

WHY LANGGRAPH AND NOT A SCRIPT

- **Conditional edges** are the decision diamonds. The flow diagram and the code stay the same shape, so neither drifts.
- **interrupt()** is the unmatched queue. The run stops, waits for a person, and resumes on the same state.
- **The checkpointer** stores every run. Replay is free, and so is the incident record.

WHAT STAYS OUTSIDE THE GRAPH

The search itself is a plain function. It takes a graph and a start id and returns a map. It has no framework in it and no network call, so it can be unit tested and run in the browser for the demo.

Demo note. On stage, run the search in the browser and keep LangGraph for the pipeline behind it. Nothing on the critical path then needs a network.

Build spec · 04b

The intake, in full.

Four steps between the text and the search. Only one step uses a model, and two checks read its output before the search runs.

#	STEP	WHO	WHAT IT DOES
1	Sniff	code	OSV JSON (`affected`) → OSV adapter. CycloneDX (`vulnerabilities`) → VEX adapter. `key: value` lines → line adapter. Anything else is prose.
2	Extract	model	Prose only. One call, temperature 0, tool forced. The prompt holds the field list from page 21 and the five examples on page 07b.
3	Gate	Jev	"Does each field exist on the target type?" "Is each version exact?" The answers and the confidence go into the receipt.
4	Resolve	code	Runs the selector on the graph. A version range becomes the exact versions the graph holds. Result: `exact`, `ambiguous` or `none`.

```
# step 2 · the only model call in the intake
EMIT_CHANGE = {
  "name": "emit_change",
  "input_schema": {
    "type": "object",
    "required": ["kind","target","where","effect"],
    "properties": {
      "kind":   {"enum": ["faulty_tool","personal_data",
                          "new_rule","other"]},
      "target": {"enum": ["tool","data","agent",
                          "workflow","obligation"]},
      "where":  {"$ref": "#/$defs/where"},
      "effect": {"$ref": "#/$defs/effect"},
      "confidence": {"type":"number","minimum":0,"maximum":1}
    }}}

resp = client.messages.create(
  model="claude-sonnet-5-5", temperature=0,
  tools=[EMIT_CHANGE],
  tool_choice={"type":"tool","name":"emit_change"},
  messages=[{"role":"user","content": raw}])
# no free text comes back: one tool call, or a failure
```

```
# added to the page 24 pipeline
g.add_node("resolve", resolve_selector) # pure
g.add_node("confirm", read_back)   # interrupt()

g.add_edge("gate_start", "resolve")
g.add_conditional_edges("resolve", match,
  {"exact":     "confirm",
   "ambiguous": "confirm",
   "none":      "triage"})
g.add_edge("confirm", "search")

# confirm passes straight through when
#   lane == "feed" and match == "exact"
# and waits for one click in every other case
```

WHEN A CHANGE FIRES ON ITS OWN

Feed · exact	Fires. The workflow's rung (page 29) still decides if it pauses.
Paste or words · exact	Card. One click, or ↵.
Any lane · ambiguous	Card with candidates. A person picks one.
Any lane · none	Unmatched queue, with the reason.

WHEN A STEP FAILS

Model is down	JSON and `key: value` still work. Prose waits in Unmatched.
Model invents a field	Schema check and Jev reject it. Unmatched.
Model picks a wrong version	Card shows "0 agents use it". Nothing fires.
Two tools share a name	Ambiguous. Radius never picks one for you.

Every parse leaves a receipt: the raw text, the tool call, the gate answers and the resolver result. The receipt joins the incident record (page 11).

Build spec · 05

The animation, to the millisecond.

One sequence, 4.3 seconds. It runs once on load and again on demand. Every value below is the one in the prototype.

START MS	ELEMENT	PROPERTY	DURATION	EASING	WHY
0	**Shockwave ring**	r 16→262 · opacity .9→0	1100	cubic-bezier(.2,.7,.3,1)	Announces the change before anything moves.
520	**Ring 1 — edge, node, label**	dashoffset len→0 · r 0→8 · opacity 0→1	500 / 420 / 400	(.4,0,.2,1) · (.2,1.5,.4,1)	Edge draws, node pops with overshoot, label last. 70 ms stagger.
1500 · 2400	**Rings 2 and 3**	same sequence	same	same	One beat per hop. The ~900 ms gap is the teaching moment.
3150	**Pause flash**	r 8.5→12→8.5	220 ×2	ease	High risk nodes only. The system acting, not reporting.
3150	**Counters**	0 → n	520	$1-(1-t)^3$	Ease-out, so the last digits settle instead of snapping.
3350 +95/row	**List rows**	opacity 0→1 · translateY 7→0	340	ease	Reads as a queue filling, not a page loading.
520–4300	**Elapsed readout**	0.0s → 3.8s	60 ms tick	linear	Counts real time. It is not a label.

Nothing reflows

Only `opacity`, `transform` and compositor-handled SVG attributes animate. Nothing reflows, so the frame rate holds on a laptop driving a projector.

Reduced motion

Under `prefers-reduced-motion` every delay becomes 0 and the final state renders at once. Nothing stays hidden behind an animation.

STAGE STATE MACHINE



The domain

Pharma is the one industry where this question is already the law.

In every other sector, “what does this change touch?” is good practice. In a regulated pharmaceutical system it is a documented obligation, and a regulator can ask to see the answer.

EU GMP Annex 11 **computerised systems**	Requires change control, now extended to configuration management. A change needs an impact assessment that covers revalidation, data integrity and business continuity. **That assessment is the Radius output.**
21 CFR Part 11 **FDA, binding**	Electronic records and signatures. Every change to a validated system still needs a documented impact assessment; only the conclusion varies.
EU GMP Annex 22 **AI and machine learning**	A new annex covering AI in GMP. Commentary expects the final text with the revised Annex 11 around mid-2026. verify before quoting
ISPE GAMP Guide: AI **July 2025, ~290 pages**	Extends GAMP 5 Appendix D11. Gives the industry a named framework for validating AI in GxP, including drift monitoring after deployment.
FDA draft guidance **January 2025**	A seven-step credibility framework scored on **model influence** × **decision consequence**. We borrow that pair as our risk rule.

The sentence that wins the room

“Annex 11 already requires an impact assessment for every change to a validated system. Most teams write it by hand, days later, from memory. Radius produces it in four seconds, with the path for each line.”

WHY A SOFTWARE TEAM CAN WIN A PHARMA TRACK

The other pharma entries will build models: in-silico trials, tissue deconvolution, literature mining. Each needs real domain science and each is hard to finish in two days.

Radius needs no pharmacology. It needs the regulation, which is written down, and a graph, which is engineering. The track asks for discovery, trials **or compliance**. This is compliance.

The pharma instance

Same engine. Higher stakes.

Nothing in the search changes. The graph contents change, one field is added, and the demo stops being about inconvenience.

WHAT CHANGES

Obligation nodes	SOX 404 → 21 CFR Part 11, Annex 11, ICH E2B, GxP validated state
Workflow nodes	Customer reply → adverse event triage, batch release review, protocol deviation
`sends_external`	→ `submits_to_regulator`
+ `validation_state`	validated, impact assessment due, requalification due

Code changed: one field and one JSON file. The search, the risk rule and the screens are untouched.

WHAT STAYS

The reverse search. The hop limit. The two-boolean risk rule. The pause. The incident. The owner message. The animation.

This is the argument for the architecture itself. A domain change is a data change. If a judge asks "would this work in finance?", the answer is a different JSON file.

THE DEMO, REWRITTEN FOR PHARMA

```
CHANGE    CVE-2026-4471 · pdf-parse 2.1.4
AGENTS    AE case intake · Batch record reviewer · Protocol deviation triage
FLOWS     Adverse event triage HIGH · Batch release review HIGH · Site monitoring LOW
AT STAKE  237 adverse event cases processed since the fault was introduced
          14 may be misclassified — the 15-day expedited clock is already running
          3 batch release decisions pending
DUTIES    ICH E2B(R3) · 21 CFR Part 11 · EU Annex 11
OUTPUT    4 validated systems need an impact assessment · 2 need requalification
```

Why this lands harder

"1,204 customer replies held" is an inconvenience. "14 adverse event cases may be misclassified and the reporting clock has been running since Tuesday" is a regulatory event with a deadline and a patient on the other end.

THE HONEST LIMIT

Radius does not decide whether a case is a SUSAR, and it must not. A qualified safety physician reviews seriousness and expectedness on every case. Radius answers one narrower question: **which cases passed through the compromised agent, and which duties are exposed.**

Review

Six objections, from people who would have to run this.

We walked the flows with two engineers, a product manager and two enterprise buyers. These questions stopped the room, and each changed the product rather than the wording.

STAFF ENGINEER

“What does “pause” actually do?”

Revoke the credential at the gateway and set `state=paused`. The orchestrator checks state before each run. **Runs already in flight finish.** We say so rather than claim a clean stop.

STAFF ENGINEER

“Your graph will be stale and you will be confidently wrong.”

Correct, and it is the real failure mode. Every header now carries `synced 4m ago` and a coverage figure. An agent with no bill-of-materials entry reads **unknown**, never safe.

STAFF ENGINEER

“Why four hops? That number looks invented.”

Fair. We ran the sweep: reach saturates at three hops and four is a margin. The numbers are on the verification page, and the limit is movable on screen.

PRODUCT MANAGER

“What happens when you pause the finance feed by mistake at month end?”

Resume asks for a reason and writes it to the record. A false pause is cheaper than a missed one only if undoing it takes seconds and leaves a trace.

ENTERPRISE CLIENT

“Nobody is letting a new tool stop production on day one.”

Agreed. Auto-pause is the fourth rung, not the first. The ladder overleaf starts at observe-only, set per workflow by its owner.

ENTERPRISE CLIENT

“We already run change management in ServiceNow. Where does this sit?”

Beside it. Radius answers the impact question and writes the result into the existing change record. Nobody moves their change process.

The objection we could not answer

“Your owner data is stale. Half our workflow owners left the company.” True, and no graph fixes it. Radius routes to the owning *team* first and names the person second, and an unowned node is surfaced as a gap on day one instead of failing silently on day ninety.

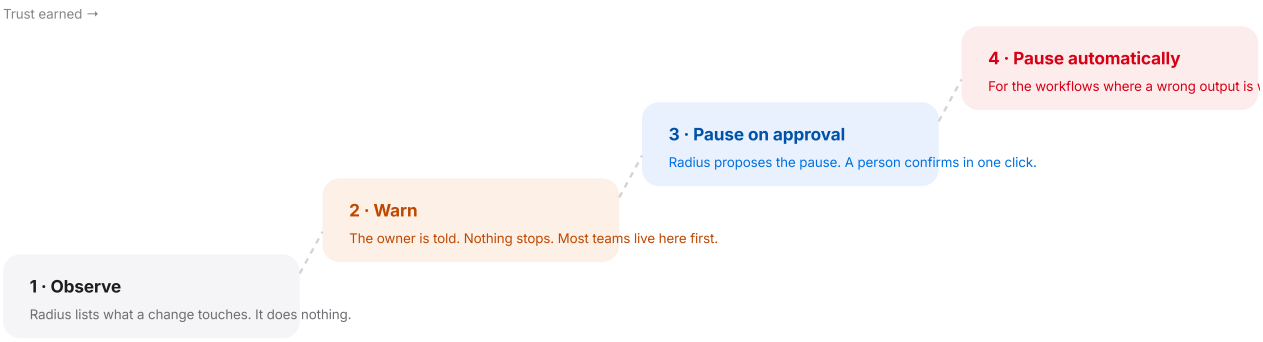
WHAT WE CHANGED AFTER THE WALKTHROUGH

- Added the adoption ladder. Auto-pause is no longer the default.
- Added graph freshness and coverage to the header of every screen.
- Resume now requires a reason, and the reason is kept.
- Made the hop limit visible and movable instead of fixed.
- Stated plainly that in-flight runs finish.

Adoption

Nobody switches this on at full power.

The rung is set per workflow by its owner. A team earns its way up as the graph proves itself. Most enterprises sit on rung two for a quarter, and that is success, not failure.



WHERE IT SITS IN AN ENTERPRISE

Radius does not replace change management. It answers the impact question that the change record already asks, and writes the answer back into that record with the paths attached.

Change raised→Radius impact list→CAB reviews→Approved

The question a pharma client asks first

"If Radius makes decisions about validated systems, does Radius itself need validating?" Yes. That is why the search is deterministic and model-free: a system that gives the same answer for the same input can be qualified against a test set. A model in the decision path could not be.

WHAT WE WOULD MEASURE AFTER NINETY DAYS

Coverage	Share of changes carrying a complete impact list, against the share written by hand today.
Time to first action	Notice to first owner step. Days to minutes is the claim; the record proves or disproves it.
False pauses	Per month. If this does not fall, the graph is wrong and no team should move up a rung.

Verification

We ran it. These are real numbers.

Every figure on this page came out of the engine, not out of a slide. The seed graph holds **32 nodes and 38 edges**. The script and the graph ship with this document.

```
$ python3 radius.py

S1  CVE-2026-4471 on pdf-parse 2.1.4
  INFECTED AGENTS    4   AE case intake · Batch record reviewer · Protocol deviation triage · Site
monitor summariser
  INFECTED WORKFLOWS 5   4 high → PAUSE, 1 low → WARN
    [PAUSE] Adverse event triage · Batch release review · Protocol deviation report · Signal review
    [warn ] Site monitoring
  DUTIES EXPOSED     3   ICH E2B(R3) · 21 CFR Part 11 · EU GMP Annex 11
  OUTPUTS AT STAKE   4   495 items

S2  patients.phone marked personal data
  2 agents · 3 workflows (3 high) · 3 duties · 9,374 items at stake

S3  New rule: models must run in the EU
  4 agents break the rule · 5 workflows affected, all high
```

HOP SWEEP — THE HONEST ANSWER TO "WHY FOUR?"

HOPS	NODES	AGENTS	FLOWS	DUTIES	OUTPUTS
1	3	4	0	0	0
2	8	4	5	0	0
3	16	4	5	3	4
4	16	4	5	3	4
5	16	4	5	3	4

Reach **stops growing at three hops** on this graph. We keep four as a margin, not because four was needed. That corrects what an earlier draft of this document claimed.

WHICH SINGLE TOOL HURTS MOST

TOOL	FLOWS	HIGH	ITEMS
pdf-parse 2.1.4	5	4	495
translate-sdk 1.4.0	2	2	**8,904**
ocr-core 3.0.2	2	2	474
libxml 2.9.1	2	1	22
pandas 2.1.0	1	1	0

The most connected tool is not the most dangerous one. pdf-parse touches more workflows, but translate-sdk sits under patient messaging and puts **18×** more items at stake.

What the simulation changed

The first run returned **zero low risk workflows**. In a pharma graph almost everything serves an obligation, so the rule marked everything high and the demo lost the warn case the brief asks for. We found that by running it, not by reading it. The graph now carries an internal workflow on the same tool, and S1 returns 4 high and 1 low.

Five minutes

The demo.

0:00

Show the graph

238 nodes. Five kinds. The judges see the whole model in ten seconds.

0:40

Send the vulnerability notice

The list appears. Say the numbers out loud: three agents, three workflows, 1,204 customer replies, one finance feed.

1:30

Let it pause the work

Two workflows pause with no human action. Two incidents open. The contract review keeps running, with a warning. Open one owner task.

2:40

Fire the second kind of change

Mark a column as personal data. A different start node, the same engine, a different list.

3:30

Hand the keyboard over

The organisers send a change nobody has seen. We add no code. The engine handles it in front of the room.

4:20

Preview a policy

Show the predicted list. Apply the rule. The two lists match.

“Other tools tell you what happened. Radius tells you what is about to happen, and then it stops it.”

Two builders. One contract. No shared file.

The work splits at the API. Builder A makes the engine that decides. Builder B makes the screens that show. Each one can build, test and finish without the other.

BUILDER A

Engine and intake

`/engine · Python 3.11 · FastAPI · LangGraph`

Decides: what the change is, what it reaches, what pauses, what each owner is told. Page 31b.

The contract

9 endpoints
7 JSON shapes
1 shared fixture

page 31d

BUILDER B

Screens and input

`/web · Vite · React · TypeScript · SVG`

Shows: the box, the feed, the blast radius, the incident, the owner message, the preview. Page 31c.

FIVE RULES THAT KEEP THEM APART

1	**One folder each**	A writes only in `/engine`. B writes only in `/web`. Only the human writes in the root.
2	**The contract is on paper**	Each builder types its own copy from page 31d: pydantic for A, TypeScript for B. There is no shared code package to fight over.
3	**B never searches**	The API returns the reach to six hops, with `hops` on each node. Actions apply at four hops. The hop slider filters the reach. It does not run a new search.
4	**A never draws**	A returns data and plain text. Colour, order on screen and timing belong to B.
5	**Only the human changes the contract**	A builder that needs a change stops and asks. The human edits page 31d and tells both builders.

CHECKPOINTS

HOUR	BUILDER A	BUILDER B
0	Both read pages 07a, 07b, 21–25, 31a–31d. The contract is frozen.	
6	S1 golden test passes from the command line.	Feed and blast radius run on the mock.
16	All endpoints answer. S1–S3 pass.	Four screens and the box run on the mock.
20	**Human:** start the engine, set `VITE_API=live`, run the demo (page 31).	
24+	Stretch: preview equals apply.	Stretch: screen 04, policy preview.

Integration is the human's task

If both builders own the join, both edit the same file. The join is one setting and one demo run. The human does it, and reports each gap as a contract change.

Builder A.
The engine and the intake.

Builder A owns everything that decides. Every task ends in a test that runs without a screen.

Folder `/engine`
Stack Python 3.11, FastAPI, LangGraph, Anthropic SDK, pytest
Reads Pages 04, 07a, 07b, 11, 18, 21–24, 24a, 29, 31d

	TASK	WHAT TO BUILD	DONE WHEN
A1	**Seed graph**	`data/graph.json` in the page 21 shape, 238 nodes. It must give the counts on pages 05 and 07. Optional: a pharma file from page 27.	S1 = 3 agents, 3 workflows, 2 high, 1 low, SOX 404. S2 = 5 agents, 7 workflows. S3 = 4 agents, 6 workflows.
A2	**Intake**	Sniffer, the OSV, VEX and line adapters, the `emit_change` extractor, the Jev gate with receipt, the resolver (page 24a).	12 sample inputs (4 per lane) in `tests/intake/` give the expected ChangeEvent. A made-up field ends in `none`.
A3	**Search and risk**	`blast_radius(graph, start_ids, max_hops=6)` from page 22, with the multi-start queue from page 07b. `risk()` from page 22.	The same input gives identical output in 100 runs. Under 10 ms on the seed graph.
A4	**Actions**	Pause or warn at ≤ 4 hops. The seven-line incident. One task for each owner. The record. Resume needs a reason. A fixed version resumes on its own.	After S1: 2 paused, 2 incidents, 3 tasks, 1 warning. Version 2.1.6 lands → both resume, tasks close.
A5	**Owner message**	The model writes the page 16 message from the incident fields only. The gate checks it. One retry, then the template.	With no API key, every task still has a message, from the template.
A6	**API**	The nine endpoints on page 31d. `mode=preview` runs the intake and the search, and skips A4. CORS open for `localhost:5173`.	The tests in `tests/contract/` pass against the S1 fixture on page 31d.
A7	**Pipeline**	The page 24 LangGraph, plus `resolve` and `confirm` from page 24a. `SqliteSaver` checkpoint.	A run stopped after `search` resumes from the checkpoint with the same result.

Not Builder A's work

No HTML, CSS, colour or animation timing. No mock data for the screens. No set-up of the demo laptop. If a task needs any of these, it belongs to B or to the human.

Hand-off

A is done when `make test` passes and `uvicorn radius.api:app` answers on port 8000. A tells the human, not Builder B.

Builder B.
The screens and the input.

Builder B owns everything a person sees. Every screen runs on the mock data from page 31d, so B never waits for A.

Folder `/web`
Stack Vite, React, TypeScript, plain SVG (no chart library)
Reads Pages 05, 06, 07a, 11–17, 12a, 19, 25, 31d

	TASK	WHAT TO BUILD	DONE WHEN
B1	**Shell**	Navigation: Changes, Graph, Policy, Record. The header shows "graph synced · coverage" from `GET /graph`.	Every screen shows freshness and coverage. Every screen is two moves from the feed (page 19).
B2	**Feed**	Wireframe 01 (page 12). Rows from `GET /changes`. Colour means state. The empty state says what was checked.	With no rows, the screen says "N changes checked · nothing paused".
B3	**Input**	Wireframe 01b (page 12a). The box, the card in three states, chips edited with values from `GET /graph`, Fire and Preview.	The three mock states render. ↵ fires only when the state is `exact`.
B4	**Blast radius**	Wireframe 02 (page 13). Rings from `hops`. The reveal on page 25, to the millisecond. The hop slider filters by `hops`. Reduced motion.	The S1 mock settles at 3.8 s. At slider 2, the outputs and duties are hidden.
B5	**Incident and queue**	Wireframe 03 (pages 15–16). Seven fixed lines. The owner queue. `/task/:id` opens the page 16 message with no navigation.	`/task/t_finance_feed` opens directly. A row leaves the queue when its task closes.
B6	**Policy preview**	Stretch, page 17. A draft rule → `POST /changes` with `mode=preview`, then apply. Show both lists and their difference.	Same lists on the mock → "LISTS MATCH". A forced difference shows the extra item.
B7	**Mock layer**	`src/mocks/` typed from page 31d. One switch: `VITE_API=mock|live`. Mock delay 300 ms.	With `VITE_API=live` and the engine on port 8000, no screen code changes.

Not Builder B's work

No search, no risk rule, no parsing of the text in the box. B sends the text and shows what comes back. B does not write `graph.json` or any file in `/engine`.

Hand-off

B is done when `npm run build` passes and all four screens and the box run on the mock. B tells the human, not Builder A.

The contract. Frozen at hour zero.

Nine endpoints, seven shapes and one example. Both builders type from this page. If this page and the code disagree, the page wins.

ENDPOINT	BODY	RETURNS	USED BY
`GET /graph`	—	`nodes, edges, synced_at, coverage`	B1 header · B3 chips
`POST /changes/parse`	`{ text }`	`ParsedChange`	B3 read-back card
`POST /changes`	`{ change, mode }`	`Run`	B3 Fire / Preview · B6
`GET /changes`	—	`FeedRow[]`	B2 feed
`GET /runs/{id}`	—	`Run`	B4 · B5
`POST /workflows/{id}/resume`	`{ reason }`	`Verdict`	B4 “Resume…”
`GET /tasks/{id}`	—	`Task + Incident`	B5 deep link
`POST /tasks/{id}/accept`	—	`Task`	B5 “I will do this”
`POST /tools/{id}/version`	`{ version }`	`Verdict[]`	Demo: the fix lands

```
// shapes · ChangeEvent is on page 07b
ParsedChange { change: ChangeEvent,
  state: "exact"|"ambiguous"|"none",
  matches: id[], candidates: {id,label,score}[],
  readback: str, reason?: str }
Run { run_id, mode, elapsed_ms, change, counts,
  reached: Reached[], verdicts: Verdict[],
  incidents: id[], tasks: id[] }
Reached { id, type, name, hops,
  path: id[], via: edge_type[] }
Verdict { workflow_id, risk: "high"|"low",
  action: "paused"|"warned"|"none" }
Incident { id, cause, path, risk,  // page 11
  duty, action, owner, next }
Task { id, owner_id, workflow_id, incident_id,
  message, state: "open"|"accepted"|"closed" }
FeedRow { change_id, title, kind, reach,
  state, at }
```

```
// fixtures/run_S1.json · trimmed · page 05
{ "run_id": "run_S1", "mode": "live", "elapsed_ms": 3800,
  "change": { /* page 07b */ },
  "counts": { "agents": 3, "workflows": 3, "paused": 2,
    "warned": 1, "duties": 1, "outputs": 2, "untouched": 60 },
  "reached": [
    { "id": "tl_pdf_parse_214", "type": "tool", "hops": 0 },
    { "id": "ag_invoice_extractor", "type": "agent", "hops": 1,
      "path": ["tl_pdf_parse_214", "ag_invoice_extractor"] },
    { "id": "wf_finance_feed", "type": "workflow", "hops": 2 },
    { "id": "ob_sox_404", "type": "obligation", "hops": 3 },
    /* … every node, each with path and via */ ],
  "verdicts": [
    { "workflow_id": "wf_customer_reply",
      "risk": "high", "action": "paused" },
    { "workflow_id": "wf_finance_feed",
      "risk": "high", "action": "paused" },
    { "workflow_id": "wf_contract_review",
      "risk": "low",  "action": "warned" } ],
  "incidents": ["INC-2041", "INC-2042"],
  "tasks": ["t_finance_feed", "t_customer_reply",
            "t_contract_review"] }
```

One fixture, two uses

B loads `run_S1.json` as mock data. A asserts that its live S1 run has the same shape and the same counts.

Add, never rename

A new optional field is safe. A rename or a removed field is a new version of this page, 31d-v2.

IDs are stable

IDs come from `graph.json` and never change in a run. B can key the animation on them.

How we designed it

Empathy. Focus. Impute.

Three lines written for Apple in 1977. They still decide every argument we had about this product.

Empathy

"We will truly understand their needs better than any other company."

The person who must act is not the person who runs the system. Sam owns the finance feed and has no interest in graphs. So Sam never opens Radius. Sam gets one message, with the cause and one next step, in plain words. We moved the work to the system and left the decision with the person.

Focus

"We must eliminate all of the unimportant opportunities."

We cut a query language, a graph explorer, a chat box and a second risk model. Each one added power. Each one also added a question that a person must answer before the product helps them. Five node kinds remain. One search remains. A judge can hold the whole model in mind.

Impute

"People DO judge a book by its cover."

Radius asks a company to let software pause its customer replies and its finance feed. No team gives that permission to a product that looks unfinished. The care in this document is not decoration. It is the first evidence that the control is safe to trust.

The one rule we never broke

If a screen needs a sentence of explanation, the screen is wrong. We changed the screen every time, and we never added the sentence.

WHAT WE REMOVED, AND WHAT IT COST

Query language	Power for an expert. A blank box for everyone else.
Chat box	An answer that changes between runs cannot hold a pause switch.
Second risk model	More accuracy. No way to explain one decision to an auditor.

WHAT WE KEPT, AND WHY

One list. One graph view. One message to an owner. One switch. A person can learn the product in a minute and use it under pressure.

Pressure is the real test. A team opens this screen on the worst morning of the quarter. Every removed feature is one less thing to think about.

Essential design principles

Each principle, one decision.

Twelve principles. Twelve choices we can defend. Nothing below is a style note. Each one changed what the product does.

Humanity

Designing with humanity in mind

Sam owns a workflow, not a security tool. Sam gets one message, in the tool Sam already uses. Radius serves the person. The person does not serve Radius.

Simple easy, complex possible

Both, in one product

The feed answers “what is wrong?” in two seconds. The graph answers “why exactly?” for the person who must prove it.

Predictability

The same input gives the same output

No model decides the list. The same change always returns the same answer. A person can test the system and trust the test.

Clean & simple

Remove, then remove again

Five node kinds. Four link kinds. We removed the query language, the graph explorer and the chat box. Each one added power and cost clarity.

Feedback

Say what the system did

The heading reads PAUSED AUTOMATICALLY. Radius reports a decision. It does not ask for one, and it never stays silent.

Consistency

One format for every event

Every pause names its cause in the same shape: the change, the path, the owner, the next step. A person learns the format once.

Affordance

Show what a person can do

The hop limit is a control that a person can move. The pause switch looks like a switch. Nothing important hides in a menu.

Help people avoid errors

Prevent, do not warn

Radius pauses the workflow before the bad output leaves. An error message after a breach is not a control. This is the principle the whole product rests on.

Visibility of system status

Report on a quiet day too

On a day with no faults the feed says how many changes it checked, and that nothing is paused. A control that shows nothing looks broken.

Progressive disclosure

Count first, path on demand

The reach is on the row. The full path opens with one tap. A person never reads more than the moment needs.

Symmetry

Pause and resume are the same shape

Every action has a matching way back, in the same place and the same form. The graph mirrors the list, item for item.

Delightful

The four second answer

A question that took three days now takes four seconds. Delight here is not an animation. It is the relief of a correct answer, at once.

Sources and language

Where each claim comes from.

Vendor figures are marked. Check any number before you say it to a judge.

RESEARCH

- **Formal Analysis and Supply Chain Security for Agentic AI Skills** · arXiv 2603.00195 · Mar 2026. The reverse breadth-first search that Radius uses.
- **AgentFlow: Agent Dependency Graphs** · arXiv 2607.01640.
- **Graph-Propagated Impact Analysis** · arXiv 2606.18855. States the limit of a structural search.
- **Trust-Aware Multi-Agent Traceability** · arXiv 2606.17203. Impact as nodes within h hops.
- **ATAG: Threat Assessment with Attack Graphs** · arXiv 2506.02859.
- **AIBoMGen** · arXiv 2601.05703 · CAIN 2026.
- **DAGGER** · arXiv 1301.0977. Background on reachability indexes.

STANDARDS

- **SBOM for AI: Minimum Elements** · seven national agencies with the European Commission · 12 May 2026.
- **CycloneDX ML-BOM** (OWASP) and **SPDX 3.0** (Linux Foundation). Neither is the single standard yet.
- **EU AI Act** Art. 11 and Annex IV. dates disputed One source gives 2 August 2026. Another claims a delay to December 2027 and August 2028. Check EUR-Lex.

PRODUCTS

- **ServiceNow** · the Analyze impact workflow and the `cmdb_rel_ci` traversal.
- **Zenity**, **Noma Security** ($100M Series B), **Microsoft Agent 365** (available 1 May 2026, $15 per user per month), **ServiceNow AI Control Tower**, **Credo AI**, **IBM watsonx.governance**.
- **Collibra**, **Atlan**, **Microsoft Purview**, **OpenLineage** with Marquez.
- **Kubernetes Gatekeeper** `dryrun`, **AWS IAM Access Analyzer** access previews, **Enterprise OPA** Live Impact Analysis.
- Cisco acquired **Astrix Security** for a reported $400M, May 2026.

FIGURES VENDOR-DERIVED

- 30% rise in change failure rates · Riftmap.
- 60% of companies cannot stop a bad agent · Kiteworks.
- 30× fewer tokens for a graph lookup · Meta, April 2026, reported by Riftmap.
- The term entered wide use after an Amazon memo in March 2026, reported by the Financial Times.

About the language in this document

The body text follows **ASD-STE100 Simplified Technical English** at about 80 percent. This is the controlled language that aerospace and defence use for technical documents.

Applied rules: one meaning for each word; short sentences; active voice; present tense; one idea in each sentence; no noun cluster longer than three words; the same term for the same thing on every page.

WHERE WE BREAK IT, AND WHY

The remaining 20 percent is in the headlines and in the two quoted lines. Simplified Technical English makes a procedure clear. It does not make a pitch memorable.

The product follows the same split. The screen uses plain words. The message to an owner uses plain words. Only the story that a person tells in a room is allowed to be warm.